Random Forest Classification of Single-Cell Gene Expression: Model Evaluation, Feature Selection, and Human–AI-Assisted Experimental Design

10/17/2026

Eric Nyeste

enyeste@sfsu.edu

1. Abstract

This study investigates the application of Random Forest classification to single-cell gene-expression data, emphasizing model evaluation, feature selection, and the methodological tradeoffs between predictive performance and model complexity. Using a dataset of 871 neuronal nuclei represented by 608 gene-expression features, the experiment evaluates binary classification of the e1 excitatory neuron cluster against non-e1 observations. Six Random Forest configurations were compared using out-of-bag (OOB) estimation, followed by manually implemented stratified three-fold cross-validation and Gini-based feature-importance analysis.

The highest-performing configuration, employing 100 trees and square-root feature sampling, achieved 99.66% OOB accuracy and 99.54% mean cross-validation accuracy. Subsequent experiments evaluated whether reduced feature sets could preserve classification performance. A six-gene classifier achieved 99.08% mean cross-validation accuracy, demonstrating substantial dimensionality reduction at the cost of increased false-negative classifications. Feature selection was performed independently within each cross-validation training fold to prevent evaluation-data leakage.

The study also documents the use of generative AI as an instructional and computational collaborator in experimental design, Python implementation, debugging, and interpretation. Findings illustrate both the practical capabilities and methodological limitations of supervised classification, while examining the role of human judgment in AI-assisted scientific inquiry.

Keywords: Random Forest; supervised machine learning; gene-expression classification; feature selection; cross-validation; model evaluation; human–AI collaboration.

2. Audit of Original database:

Training DB Audit Checklist

1. Documentation of description of features:
    a. Supervised-case Dataset dimensions: 871 samples, 608 gene-expression features, and one binary label column. Gene expressions are features (columns), individual nuclei are samples (rows) and the measured expression of particular genes within the nuclei constitute individual data points. (Aevermann et al., R42. 2018)
2. Class labels and ground truth evaluation:
    a. "The methodology described here is unique in that it determines the minimum number of differentially expressed genes, working in concert, that are sufficient to define a cell type from a given dataset." (R42) **Class memberships were assigned** through an iterative clustering procedure using gene-expression profiles. NSforest subsequently uses those assignments as reference labels. For this binary classification task, **ground truth** is encoded as either 1 (presumed membership in the e1 excitatory neuron cluster) or 0 (non-e1 membership).
3. Adequate and fair demography?
    a. The sample was obtained from a single post-mortem brain and is complicated because demographic characteristics and population-level generalizability **cannot be adequately assessed from the information provided.**
4. Are classes imbalanced?
    a. Of 871 samples, Class e1 (1) contains 299 samples and represents 34.3% of the data. Non-e1 (0) samples constitute the remainder of the 572 samples, 65.7% of the data. This suggests moderate class imbalance, but **neither class falls below the established 10% threshold.**
5. Type of each feature (numerical, categorical nominal or ordinal):
    a. **Features are numerical** and reflect a measured expression of a single gene within a sample.
6. Are any values missing?
    a. There are a considerable number of zeroes present in the data, which should be understood as no detected gene expression (zero value), not omissions from the dataset. Otherwise, the dataset **contains no identified missing values**.
7. Are there enough samples compared to the number of features used?

    a. No. There are 871 Samples and 608 features. This produces a ratio of **1.43 samples per feature**, well below 10:1.

8. Privacy check:
    a. **No directly personally identifiable features** are apparent in the dataset.

3. Creation of training DB and verification DB:

Method

See Appendix 3.1: Script for the Creation of Training Dataset

The original database was partitioned using stratified random sampling implemented with Python's pandas library. One observation was selected from each decision class using DataFrame.sample(n=1, random_state=42). The fixed random seed ensures reproducibility.

The two selected observations were reserved for final verification and excluded from all subsequent model training and cross-validation procedures. The original database was preserved without modification.

The resulting training and verification database statistics are summarized in Tables 1 and 2. The complete Python implementation is provided in Appendix 3.1.

The script performs the following operations:

1. Load the original CSV without modifying it.
2. Randomly select one observation from class 1 and one from class 0.
3. Combine those observations into a two-sample verification database.
4. Remove those observations from the training database.
5. Save both databases as separate CSV files.
6. Print statistics to verify the operation.

For my part, my prompts are in the Appendix and detail my thought process for how to use code to generate random sampling. My use of GenAI in this context is experimental and intended to expand my understanding of the tools involved. I could have just as easily randomized the line selection using multiple dice rolls but would not have benefitted from critical exposure to Python code generation and its pandas library. As noted in section 9, this challenge advances my reasoning beyond instrumental problem solving and expands my activity into experimental design considerations. In short, by encouraging and roadmapping *reproducibility*, it's helping me become a better scientist.

Results

Also See Appendix 3.1: Script for the Creation of Training Dataset

Table 1. Training database statistics following stratified random sampling.

Property	Value
Samples	869
Gene-expression features	608
Ground truth columns	1
Class 1 (e1)	298
Class 0 (non e1)	571
Total columns	609

Table 2. Verification database statistics following stratified random sampling.

Property	Value
Samples	2
Gene-expression features	608
Ground truth columns	1
Class 1 (e1)	1
Class 0 (non e1)	1
Total columns	609

The original database contained 871 observations and 609 columns. Two observations were withheld for verification, leaving 869 observations for training. The selected verification samples correspond to Excel rows 283 and 813 of the original database. Both CSV files were successfully generated.

4. Software tools:

ML and GenAI tools used:

Python

- Python Version: 3.14.7 (64-bit)
    - Operating system: Microsoft Windows 11 Home
    - Python environment: Python 3.14, 64-bit and IDLE Shell
    - Development interface: Python interactive interpreter (`>>>`) accessed through Windows Command Prompt
- Pandas Version: 3.0.6
    - Summary: Powerful data structures for data analysis, time series, and statistics
    - Home-page: https://pandas.pydata.org
- SciKit.Learn Version: 1.9.0
    - Summary: A set of python modules for machine learning and data mining
    - Home-page: https://scikit-learn.org
- NumPy: 2.5.3
- Matplotlib: 3.11.2

GenAI

- ChatGPT, Edu Edition.
    - GenAI platform: OpenAI ChatGPT
    - Model: GPT-6
    - Application: Conceptual instruction, Python code generation, debugging, experimental design consultation, and report review.

5. Experimental Methods and Setup

As this research is biological in nature and seeks to associate patterns of gene expression with potentially significant implications for cell-type classification, I considered an experimental approach that would minimize false-negative classifications. Specifically, a false negative occurs when a sample belonging to the e1 excitatory neuron cluster is incorrectly classified as non-e1.

The proposed experimental design will compare Random Forest configurations using two model-selection objectives: maximizing the F1 score for the e1 class and maximizing overall classification accuracy. F1 provides a balance between precision and recall, while overall accuracy measures the proportion of correctly classified observations across both decision classes. Recall measures the proportion of actual e1 observations correctly identified, allowing us to investigate whether reducing false negatives introduces a tradeoff with overall accuracy or precision.

In addition, precision, recall, and F1 will be calculated separately for both the e1 and non-e1 classes. Comparing these results will help determine whether the classifier performs more effectively for one class than the other.

These objectives will be evaluated using a common set of Random Forest configurations to permit meaningful comparison of model performance. The following sections describe the selected hyperparameters, including ntree, mtry, and classification cutoff, along with the out-of-bag (OOB) and manually implemented three-fold cross-validation (CV) procedures used to estimate classification performance.

Training dataset inputs were verified prior to model training. See Appendix 5.1.

The proposed experiment structure is as follows:

1. Primary experiment: Train six Random Forest configurations, compare their OOB F1 scores for e1, and identify the strongest candidate.
2. Secondary comparison: Evaluate accuracy, precision, recall, and F1 for both e1 and non-e1, using the same models and predictions.
3. Cross-validation: Perform manual stratified three-fold CV to compare performance estimates and assess the stability of the selected configuration. For the sake of resources, we will use the same folds for every candidate and report the per-fold results.

4. Feature ranking: Extract the ten highest-ranked genes from the selected Random Forest using Gini importance.
5. Final verification: Apply the selected model to the two withheld samples, which remain untouched until Section 8.

The proposed training parameters are:

Parameter	Selected Value	Explanation
`ntree`	100, 500, 1000	Number of trees in each forest
`mtry`	`"sqrt"`, `0.5`	Number of features considered at each split
`cutoff`	0.5	Initial and exploratory probability thresholds for classifying e1
	0.4, 0.6 (exploratory)	
`bootstrap`	`True`	Sample observations with replacement for each tree
`random_state`	42	Make results reproducible
`n_jobs`	-1	Use available CPU cores

Method 1: basic training with OOB accuracy estimate.

Six Random Forest configurations will be trained using scikit-learn's RandomForestClassifier. The experiment will vary the number of trees (n_estimators = 100, 500, 1000) and the number of candidate features considered at each split (max_features = "sqrt", 0.5). The classification cutoff will initially remain fixed at 0.5.

Bootstrap sampling will be enabled to support OOB evaluation. Each tree will train on a bootstrap sample of the training observations, leaving some observations unused by that tree.

Predictions from trees that did not train on a given observation will be aggregated to estimate its classification performance.

For each configuration, OOB predictions will be used to construct a confusion matrix and calculate accuracy, precision, recall, and F1 scores for both decision classes.

The six configurations will be ranked independently by e1-class F1 and overall accuracy. The highest-performing configuration under each criterion will be selected for comparison using manual three-fold cross-validation. If both criteria identify the same configuration, only one candidate will require further evaluation.

Software Methods:

Function or attribute	Purpose
RandomForestClassifier()	Construct the model
.fit(X, y)	Train the forest
.oob_decision_function_	Obtain OOB class probabilities
.oob_score_	Obtain default OOB accuracy
confusion_matrix()	Count TP, TN, FP, FN
accuracy_score()	Calculate accuracy
precision_score()	Calculate precision
recall_score()	Calculate recall
f1_score()	Calculate F1
classification_report()	Summarize class-specific performance

Method 2: Using 3 fold CV for accuracy estimation:

The 869 training observations will be divided into three approximately equal, class-stratified folds using a fixed random seed. The same folds will be used for all selected Random Forest configurations.

Cross-validation will be implemented manually, without using scikit-learn's automated cross-validation scoring functions.

For each configuration, three training and evaluation cycles will be performed:

1. Train on folds 2 and 3; evaluate on fold 1.
2. Train on folds 1 and 3; evaluate on fold 2.
3. Train on folds 1 and 2; evaluate on fold 3.

Each evaluation will produce a confusion matrix, accuracy, precision, recall, and F1 scores. The three fold-level results will be reported separately, and the mean and standard deviation of the performance metrics will be calculated.

The results will be compared with the OOB estimates to assess consistency between the two evaluation methods. The two previously reserved verification observations will remain excluded from all training and cross-validation operations.

-

Verification and Comparison:

Outputs from OOB tests and 3-fold cross-validation tests will be compared 1:1 on the basis of four metrics: F1 score, Accuracy, Precision, and Recall.

F1 score is calculated by doubling the number of true positives (2TP) and dividing it by itself in addition to the sum of all false positives (FP) and false negatives (FN) to achieve a score of the balance (harmonic mean) of Recall and Precision (below.) A score closer to 1 (2TP/2TP+0) is representative of data which is correctly classified relative to the supplied ground-truth labels.

Accuracy is the measure of the total of true positive and negative results (TP+TN) divided by the total dataset (TP+TN+FP+FN) to score the performance of the classifier, with a score of 1 indicating the report of only truly representative data.

Precision is the measure of true positive (TP) data divided by the sum of true positive and false positive (TP+FP) data to determine a ratio of true to false positives. False positives are the report of class membership when class membership is not warranted. In short, the classifier assigns more true values than are actual. False positives can cause unnecessary procedures, costs, and anxiety. The patient trades off being extra diligent without cancer, rather than failing to take measures to mitigate a false negative. The example is hypothetical and intended to demonstrate the difference between optimizing for precision versus recall.

Recall is the measure of true positive (TP) data divided by the sum of true positive and false negative data (TP+FN) to determine the proportion of all actual positive observations that were correctly identified. False negatives occur when an actual positive result is classified as negative, which can have significant real world implications including the death of a patient in the aforementioned cancer screenings where time is critically of the essence.

Output comparison between OOB and the mean of three, manual CV folds will compare the above metric as well as their respective Confusion Matrices to scan for meaningful difference in reporting. Those variances will then be reported and compared against subsequent tests to develop relative measures of parameter effectiveness and stable consistency for ML predictions. As noted in the parameters section above, cutoff thresholds of 0.4 and 0.6 are reserved for subsequent comparative testing in just such a case.

6. Results of RF Training and Accuracy Estimates:

Method 1: Basic training with OOB Estimate

The six Random Forest configurations were trained and evaluated using Python and scikit-learn. The complete executable script, including hyperparameter iteration, OOB prediction, performance calculations, and CSV export, is provided in Appendix 6.

Key Findings:

- The 100-tree, square-root configuration achieved the highest OOB accuracy and e1 F1 score.
- Increasing the number of trees did not consistently improve performance.
- Square-root feature sampling outperformed 50% feature sampling across all three forest sizes.
- Non-e1 F1 was evaluated for the sake of comparison to e1 data. NOTE: The current ranking identifies the same winner, regardless of class.

Figure 6.1: OOB Accuracy by Random Forest configuration

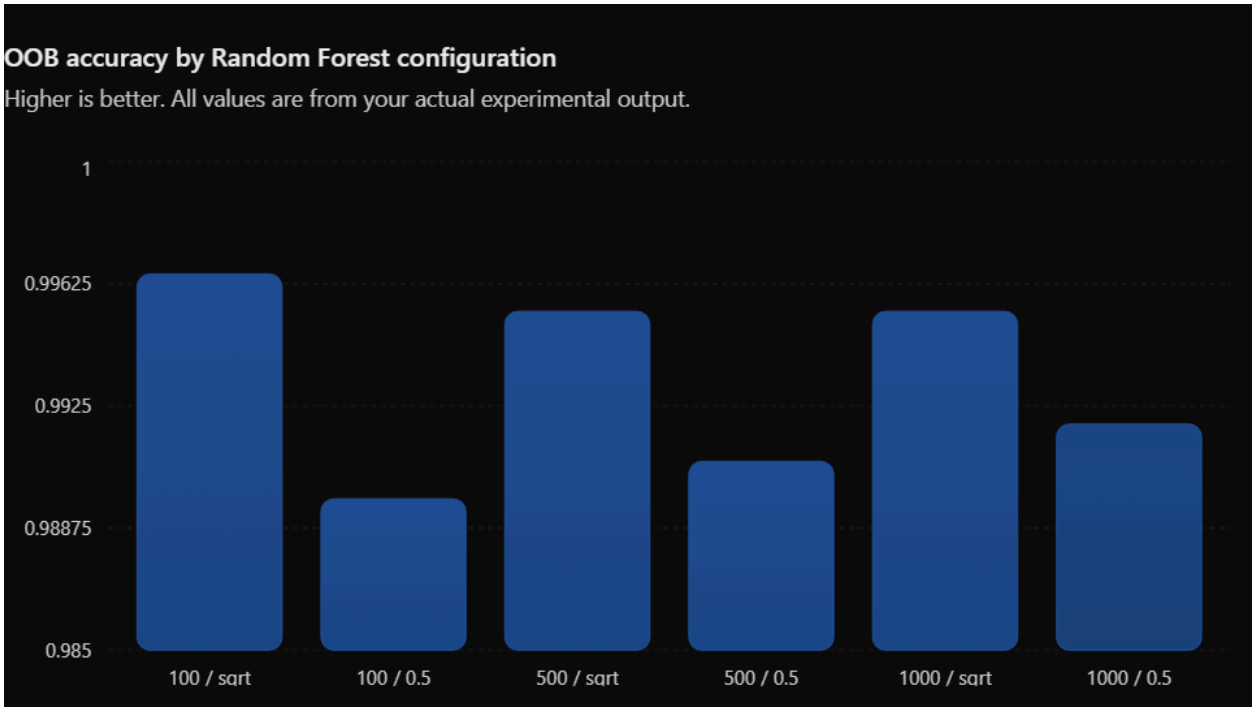


Figure 6.1. Out-of-bag classification accuracy for six Random Forest configurations trained on 869 observations with 608 gene-expression features. Forest size varied from 100 to 1,000 trees, while candidate-feature selection used either the square root of the feature count or 50% of the available features.

Table 6.1: Six-Pass Comparison Data

ntree	mtry	OOB accuracy	e1 F1	Non-e1 F1	Errors
100	sqrt	0.99655	0.99497	0.99737	3
100	0.5	0.98964	0.98497	0.99210	9
500	sqrt	0.99540	0.99331	0.99649	4
500	0.5	0.99079	0.98667	0.99297	8
1000	sqrt	0.99540	0.99331	0.99649	4
1000	0.5	0.99194	0.98831	0.99385	7

Table 6.1. Comparison of OOB accuracy, e1 and non-e1 F1 scores, and total classification errors across six Random Forest configurations. All models were trained using 869 observations and evaluated with a classification cutoff of 0.5.

Table 6.2: Best-performing Random Forest, OOB evaluation

Overall OOB accuracy: 0.99655

Metric	e1 (Class 1)	non-e1 (Class 0)
Precision	0.99331	0.99825
Recall	0.99664	0.99650
F1 score	0.99497	0.99737
Support	298	571

Table 6.3: Confusion Matrix

100-tree Random Forest, mtry = sqrt

Rows: actual class · Columns: predicted class

	Predicted non-e1 (0)	**Predicted e1 (1)**
Actual non-e1 (0)	569 (TN)	2 (FP)
Actual e1 (1)	1 (FN)	297 (TP)

Table 6.3 demonstrates the count of False Positives (FP) in actual non-e1 vs predicted e1 data and False Negatives (FN) in actual e1 class data vs predicted non-e1 class data. This shows that predictions have a lower instance of FN than FP and very low rate overall.

Method 2: Basic training with 3-Fold Cross Validation

The complete script is available in Appendix 6.3

The complete 3-Fold Cross Validation script follows the experimental design. It:

- Uses the winning Random Forest configuration: 100 trees, `mtry="sqrt"`.
- Manually constructs three class-stratified folds.
- Trains three separate Random Forest models.
- Calculates accuracy, precision, recall, and F1 for both classes.
- Generates a confusion matrix for each fold.
- Calculates the mean and standard deviation of the three folds' metrics.
- Produces one pooled confusion matrix across all 869 observations.
- Saves the results as CSV files for Section 6.
- Does not use scikit's automated cross validation method and preserves the training data.

Key Findings:

- The CV procedure produced one additional false positive, while the number of false negatives remained unchanged.
- It produced a mean accuracy of 0.99540.
- It produced a mean e1 F1 score of 0.99332.

Table 6.4 Results of 3 Fold Cross Validation

Fold	Training samples	Evaluation samples	Accuracy	e1 F1
1	578	291	0.99313	0.99000
2	580	289	0.99654	0.99497
3	580	289	0.99654	0.99497
Mean			0.99540	0.99332
Std. deviation			0.00197	0.00287

Table 6.5 Pooled Confusion Matrix

Pooled CV

	Predicted, non-e1 (0)	Predicted, e1 (1)
Actual, non e1 (0)	568 (TN)	3 (FP)
Actual, e1 (1)	1 (FN)	297 (TP)

Verification and Comparison

The results support three observations:

- Both evaluation methods achieve exceptionally high accuracy, above 99.5%.
- Both identify the same proportion of e1 observations correctly, with one false negative each.
- The small difference in overall performance comes from false-positive classifications, not additional missed e1 observations.

These findings support the consistency of the classifier's performance estimates within the supplied dataset.

They do not establish that the model would perform equally well on tissue from other donors, brain regions, or sequencing protocols.

There's also a methodological limitation worth recording: the 100-tree configuration was selected using OOB results from these same 869 observations. Consequently, the CV comparison is informative, but it is not an entirely independent evaluation of model selection.

OOB versus cross-validation

Selected Random Forest · 100 trees · mtry = sqrt

OOB accuracy

0.99655

Pooled CV accuracy

0.99540

Difference: 0.00115, or approximately 0.115 percentage points.

Table 6.6 Comparison chart of OOB vs 3-Fold Cross Validation

Metric	OOB	Pooled 3-fold CV	Difference (CV – OOB)
Accuracy	0.99655	0.99540	-0.00115
e1 precision	0.99331	0.99000	-0.00331
e1 recall	0.99664	0.99664	0.00000
e1 F1	0.99497	0.99331	-0.00166
non-e1 F1	0.99737	0.99649	-0.00088

Table 6.6 demonstrates key comparative differences in OOB vs 3-Fold Cross Validation methods.

7. Feature Ranking and Explainability:

Objectives of Feature Ranking

1. Train the selected forest on all 869 training observations.
2. Extract and rank the 608 features by Gini importance.
3. Display and graph the top ten genes.
4. Compare those genes with the paper's markers: TESPA1, LINC00507, SLC17A7, and KCNIP1.
5. Evaluate whether a smaller number of genes can preserve classification performance.

Method for Obtaining Feature Ranking

See Appendix 7.1 for full feature ranking script

Using ChatGPT, we developed a script to rank all features by GINI importance and output the top 10 to a graphical chart.

The script performs:

1. Training of the selected 100-tree Random Forest.
2. Prints the ten highest-ranked genes.
3. Reports the rankings of the four published e1 marker genes.
4. Generates and display a bar chart (below.)
5. Saves the chart and complete feature rankings as a separate file.

Figure 7.1: Feature-ranking results

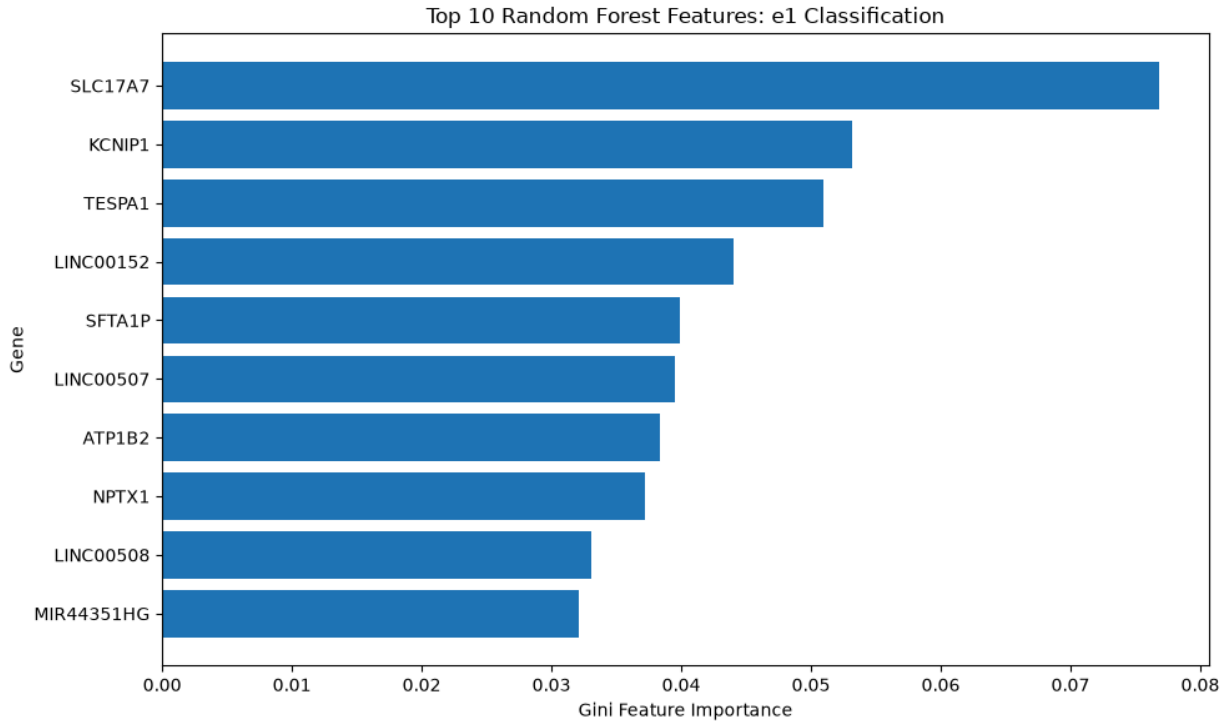


Figure 7.1 is a bar-chart representation of gene ranking listing SLC17A7 at the top.

Table 7.1: Feature Ranking Numerical Results

Rank	Gene	Gini importance
1	SLC17A7	0.07683
2	KCNIP1	0.05316
3	TESPA1	0.05093
4	LINC00152	0.04402
5	SFTA1P	0.03994
6	LINC00507	0.03953
7	ATP1B2	0.03832
8	NPTX1	0.03725
9	LINC00508	0.03311
10	MIR44351HG	0.03209

Table 7.1: Gini-based, numerical feature importance from the selected 100-tree Random Forest trained on 869 observations with 608 gene-expression features.

Initial Results

As a result of running the script on the training dataset, we can observe that SLC17A7 ranks first and all four published markers appear in the top six. Gini importance measures predictive contribution rather than biological causation.

Reduced Feature Comparison Testing

Reduced Feature Comparison Script available in Appendix 7.2
Full results available in Appendix 7.3

- The full model wins on accuracy, F1, and minimizing false negatives.
- The six-feature model achieves approximately 99.08% mean CV accuracy while using only six of the original 608 genes, a reduction of approximately 99% in feature count.
- However, its mean e1 F1 falls from 0.99332 to 0.98660, and it misses four e1 observations rather than one.
- The ten-feature model doesn't improve overall accuracy over the six-feature model. That's not necessarily surprising: adding features doesn't guarantee better predictions, particularly when correlated genes provide overlapping information.
- The appearance of all four published markers among the six highest-ranked features provides evidence that the classifier relies on biologically relevant expression patterns.
- Agreement with the paper increases confidence in the model's predictive relevance, but does not independently validate the biological classifications.

Methodological clarification: Feature selection and cross-validation
The feature rankings presented in Table 7.1 were generated using the complete training dataset of 869 observations and are provided for exploratory interpretation and comparison with published biological markers.

For the reduced-feature cross-validation experiments, feature importance was independently recalculated using only the training observations within each fold. Feature subsets were selected from these fold-specific rankings before evaluation on the corresponding held-out observations. This procedure prevents evaluation-fold information from influencing feature selection.

The comparison of feature counts remains exploratory, as the reported cross-validation results also informed the selection of the final six-feature configuration.

Table 7.2: Reduced-feature comparison

Features	Mean CV accuracy	Mean e1 F1	False negatives
4	0.98847	0.98323	4
6	0.99080	0.98660	4
10	0.98965	0.98501	3
608	0.99540	0.99332	1

Table 7.2 shows mean accuracy and e1 F1 are averaged across three manually constructed CV folds. False negatives are summed across the three held-out folds.

Observations for future consideration

- All four published e1 markers appear among the top six genes in the full-data Gini ranking.
- Reducing the feature count from 608 to six preserves high classification accuracy, but causes a measurable decline in e1 F1 and recall.
- The six-feature configuration provides the strongest observed compromise between model simplicity and predictive performance.
- The 608-feature configuration remains preferable when minimizing false negatives is the priority.
- Gini importance describes the contribution of features to tree splitting, not biological causality or necessity.

8. RF Run Time test:

Feature Selection and Justification:

In Section 5, I pointed to the importance of minimizing false negatives given the connection between gene-expression classification and its potential application to biomedical research. The Reduced Feature Comparison provided an important consideration for the development of the Runtime Engine. Referring to Table 7.2, the full 608-feature model produces the fewest false negatives, while each of the reduced-feature configurations produces more. However, the six-feature configuration achieves the highest F1 score among the reduced-feature models while reducing the number of required features by approximately 99%.

I will therefore select the six-feature configuration for the final Runtime Engine, accepting a modest reduction in classification performance in exchange for substantially lower feature requirements. The full 608-feature model may also be evaluated as a secondary comparison, without using the verification results to revise the original model-selection decision.

Table 8.1 Reduced Feature Comparison Data

Features	Mean accuracy	Mean e1 F1	False negatives
4	0.98847	0.98323	4
6	0.99080	0.98660	4
10	0.98965	0.98501	3
608	0.99540	0.99332	1

Final model configuration

Refer to Appendix 8.1 for full RTE code

Table 8.2 Selected Parameters

Parameter	Selected value
Algorithm	RandomForestClassifier
Number of trees	100
max_features	"sqrt"
Predictive features	6 genes
Training observations	869
Verification observations	2
Classification cutoff	0.5
Random seed	42

The script will produce a table containing:

Table 8.3 Results Key

Output	Meaning
Ground truth	Known class membership from the verification database
Predicted class	The class selected by Random Forest
P_non_e1	Estimated probability of non-e1 membership
P_e1	Estimated probability of e1 membership
Correct	Whether the prediction matches ground truth

Table 8.4. Runtime verification results

Ground truth	Predicted class	P(non-e1)	P(e1)	Correct
1 (e1)	1 (e1)	0.00000	1.00000	Yes
0 (non-e1)	0 (non-e1)	1.00000	0.00000	Yes

The final runtime verification correctly classified both withheld observations, assigning a class-probability estimate of 1.00000 to the corresponding ground-truth class in each case. While these results demonstrate successful classification of the two verification samples, they do not establish the model's generalizable accuracy or biological certainty.

Further inspection of individual tree predictions and the precision of the reported probabilities could clarify how the ensemble arrived at these results. Verification of the separation between training and holdout observations would provide an additional safeguard against data leakage.

The results are therefore encouraging within the scope of this experiment, but the limited verification sample size prevents broader conclusions about predictive reliability.

9. Use of GenAI:

Overview

I have been both fully in conversation with OpenAI Chat GPT-6 and substantially out of depth as a philosopher for much of this assignment. The insights that such an assignment have produced are deeply interesting to me and hopefully will assist in the development of new epistemic tools to gauge the effects of AI use on individuals. Below is a breakdown that was provided for me by my research partner (ChatGPT). A new feature of the model is the option to export a full .pdf transcript of my interaction with the chatbot: an artifact which I find to be particularly useful for the next round of data projects.

GenAI contribution	What happened	Suggested usefulness
Conceptual instruction	Clarified supervised classification, ntree, mtry, OOB, CV, and evaluation metrics	HIGH
Python development	Generated scripts for sampling, Random Forest training, manual CV, feature ranking, and runtime inference	HIGH
Debugging	Helped resolve installation, file-path, syntax, and execution problems	HIGH
Experimental interpretation	Assisted with comparing performance, selecting features, and identifying methodological limitations	HIGH

By contrast, throughout this project, ChatGPT recognized that I made several substantive experimental decisions, including:

- Proposing stratified random sampling and considering reproducibility.
- Selecting the hyperparameter ranges with reference to the research paper.
- Prioritizing false-negative classifications and comparing class-specific performance.
- Insisting that feature selection and final verification remain separate experiments.
- Selecting the six-feature runtime classifier despite its slightly lower accuracy.

ChatGPT contributed explanations, implementation suggestions, executable code, debugging assistance, and interpretation. That division of labor is important. It demonstrates meaningful human involvement without understating how extensively GenAI assisted in producing the code.

Clearly, I would not have been able to produce this report without the use of GenAI. While I am substantially indebted to it, I now also have a deep respect for the information that I have been exposed to over the last 8 weeks. Without Professor Petkovik's immense slide base and deep understanding of ML tools, I would not have been able to converse with the model, argue, interject, and challenge its responses, arguments, and strategies. I found myself often switching between Python and ChatGPT and asking myself, "has it already calculated out all of the things that I am asking for and is just toying with me? It is very enthusiastically complimentary when I appear to be learning.

Its worth noting that I have been using ChatGPT over the past 6 weeks to learn Python and our lessons are certainly part of its memory. So a significant character has presented itself within my interactions. I am not just coding with a machine: I'm coding with a familiar pedagogical, thought, and work partner. In the below example, I had interjected about random sampling with dice versus running a script as a plausible means. It used a familiar pattern to analyse my objection and respond in a way that I find useful for learning:

EXAMPLE: On Randomizing Methodology From Section 3

Your comparison to dice rolls raises an interesting distinction.
Randomness is not the same as reproducibility.
Rolling dice could produce a perfectly valid random selection. But unless you recorded the results, another researcher couldn't reproduce the exact partition.
Using random_state=42 allows the same sampling operation to produce the same observations when applied to the same dataset under the same conditions.
That's a useful lesson in experimental design, independent of the sophistication of the algorithm.

In conclusion, I have so many more questions. But I have received a glimpse of the scope of the power of the tool and of how much that I don't understand. I had to remember how to think like a scientist and also be ethical in a morally hazardous arena: ChatGPT is not on a timeframe, but I exist on a deadline. The opportunity to cheat is very real and must be further scrutinized.

10. Resources:

Research Paper

Aevermann, B. D., Novotny, M., Bakken, T., Miller, J. A., Diehl, A. D., Osumi-Sutherland, D., Lasken, R. S., Lein, E. S., & Scheuermann, R. H. (2018). Cell type discovery using single-cell transcriptomics: Implications for ontological representation. *Human Molecular Genetics, 27*(R1), R40–R47. https://doi.org/10.1093/hmg/ddy100

Machine Learning and Scientific Computing Resources

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 51–56). https://doi.org/10.25080/Majora-92bf1922-00a

NumPy Developers. (n.d.). *NumPy documentation*. https://numpy.org/doc/stable/

pandas development team. (n.d.). *pandas documentation*. https://pandas.pydata.org/docs/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

Python Software Foundation. (n.d.). *Python 3 documentation*. https://docs.python.org/3/

Generative Artificial Intelligence

OpenAI. (2026). *ChatGPT (GPT-6, Edu edition)* [Large language model]. https://chatgpt.com/

Application: Conceptual instruction, Python code generation, debugging, experimental-design consultation, statistical interpretation, and report review. Specific prompts, generated scripts, and examples of human oversight are documented in Section 9 and the appendices.

Software Environment

The experiments were conducted using Python 3.14.7 (64-bit) on Microsoft Windows 11 Home, with scikit-learn 1.9.0 and pandas 3.0.6. NumPy supported array manipulation and manual cross-validation; Matplotlib supported graphical representation of feature rankings.

Python IDLE and Windows Command Prompt were used for script development, execution, debugging, and package installation.

Human Collaboration

No group members or other students contributed to the execution, analysis, or preparation of this assignment. The work was completed individually with the disclosed assistance of OpenAI ChatGPT.

11. Appendix

3.1 Script for the Creation of Training Database

The script will:

7. Load the original CSV without modifying it.
8. Randomly select one observation from class `1` and one from class `0`.
9. Combine those observations into a two-sample verification database.
10. Remove those observations from the training database.
11. Save both databases as separate CSV files.
12. Print statistics to verify the operation.

Source: ChatGPT, edu edition.

Prompts:

1.) "so for rows 2-300 choose a random number and for rows 301-872 choose a random number. is this a sufficiently rigorous method?"

2.) "so would our Python use a code line like: if 'label'=1: Yes(random.randint)"

3.) "Okay, so lets produce the code required to produce the random dataset and ill place it in the Appendix"

```
(/) Python
import pandas as pd

# Load original database
df = pd.read_csv("Original training DB e1 cluster.csv")

# Set random seed for reproducibility
seed = 42

# Randomly select one sample from each decision class
sample_1 = df[df["Label"] == 1].sample(
    n=1, random_state=seed
)

sample_0 = df[df["Label"] == 0].sample(
    n=1, random_state=seed
)

# Combine selected samples into verification database
verification_db = pd.concat([sample_1, sample_0])

# Remove verification samples from training database
training_db = df.drop(index=verification_db.index)
```

```
# Save both databases as separate CSV files
verification_db.to_csv("verification_db.csv", index=False)
training_db.to_csv("training_db.csv", index=False)

# Display database statistics
print("Original database:", df.shape)
print("Training database:", training_db.shape)
print("Verification database:", verification_db.shape)

print("\nTraining class distribution:")
print(training_db["Label"].value_counts().sort_index())

print("\nVerification class distribution:")
print(verification_db["Label"].value_counts().sort_index())

# Record original Excel row numbers (header is row 1)
print("\nSelected original Excel rows:")
print(verification_db.index + 2)

# Confirm no samples were lost or duplicated
assert len(training_db) + len(verification_db) == len(df)
assert training_db.index.intersection(verification_db.index).empty
```

Resulting in:

```
Original database: (871, 609)
Training database: (869, 609)
Verification database: (2, 609)

Training class distribution:
Label
0    571
1    298
Name: count, dtype: int64

Verification class distribution:
Label
0    1
1    1
Name: count, dtype: int64

Selected original Excel rows:
RangeIndex(start=283, stop=1343, step=530)
```

5.1 Training Dataset Input Verification

Prior to training the model, Chat GPT recommended that we validate the model inputs on the Training_DB.csv file that was generated by the previous script. No specific prompt was used to engage this suggestion.

```
>>> import pandas as pd
...
... training_db = pd.read_csv(folder / "training_db.csv")
... verification_db = pd.read_csv(folder / "verification_db.csv")
...
>>> X = training_db.drop(columns=["Label"])
... y = training_db["Label"]
...
... print("Feature matrix:", X.shape)
... print("Target vector:", y.shape)
... print("Classes:", sorted(y.unique()))
... print("Missing values:", X.isna().sum().sum())
...
Feature matrix: (869, 608)
Target vector: (869,)
Classes: [np.int64(0), np.int64(1)]
Missing values: 0
>>>
```

The results validate 869 samples, 608 features, two ground truth classes, and no missing data values.

5.2 Parameter Selection

ChatGPT Prompt:

So, I like the idea of using an NTREE of 1000 like in the research paper and accept your suggestion that we can go lower, not necessarily higher. So 500 and 100 are also good. MTRY same, lets use the sqrt function as specified in the research paper. I accept your suggestion to use 0.5 as well (pure average?) I assume that the cutoff of 0.5 has to do with there only being two classes.

One other question, should we run these tests for the non-e1 class and compare? would that be more productive than running the optimized recall pass?

6.1 RF Train on NTREE 100, MTRY “sqrt”

```
from sklearn.ensemble import RandomForestClassifier
print ("Random Forest is ready!")

import pandas as pd

training_db = pd.read_csv(
    r"C:\Users\enyes\OneDrive\Documents\training_db.csv")

print("Training Database Loaded!")
print("Dimensions:", training_db.shape)

# Separate predictive features from ground-truth labels
X = training_db.drop(columns=["Label"])
y = training_db["Label"]

print("Features:", X.shape)
print("Labels:", y.shape)

rf = RandomForestClassifier(
    n_estimators=100,
    max_features="sqrt",
    bootstrap=True,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)

print("Random Forest configured successfully!")

# Train the Random Forest classifier
print("Training Random Forest...")

rf.fit(X, y)

print("Random Forest training complete!")

# Report out-of-bag accuracy
print("OOB Accuracy:", rf.oob_score_)

# Display OOB Decision Metrics
from sklearn.metrics import confusion_matrix, classification_report

# Convert OOB probabilities into predicted classes
oob_predictions = rf.oob_decision_function_.argmax(axis=1)

# Compare predictions against ground truth
cm = confusion_matrix(y, oob_predictions, labels=[0, 1])

print("\nOOB Confusion Matrix:")
print(cm)

print("\nOOB Classification Report:")
print(classification_report(y, oob_predictions, digits=5))
```

6.2 RF Train on 6 Passes: NTREE [100,500,1000], MTRY [“sqrt”, 0.50]

```
from sklearn.ensemble import RandomForestClassifier
print ("Random Forest is ready!")

import pandas as pd

training_db = pd.read_csv(
    r"C:\Users\enyes\OneDrive\Documents\training_db.csv")

print("Training Database Loaded!")
print("Dimensions:", training_db.shape)

# Separate predictive features from ground-truth labels
X = training_db.drop(columns=["Label"])
y = training_db["Label"]

print("Features:", X.shape)
print("Labels:", y.shape)


# Train Random Forest using six hyperparameter configurations

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

results = []

for ntree in [100, 500, 1000]:
    for mtry in ["sqrt", 0.5]:

        print("\nTraining:", ntree, "trees, mtry =", mtry)

        # Configure and train a new forest
        rf = RandomForestClassifier(
            n_estimators=ntree,
            max_features=mtry,
            bootstrap=True,
            oob_score=True,
            random_state=42,
            n_jobs=-1
        )

        rf.fit(X, y)

        # Generate OOB predictions
        oob_predictions = rf.oob_decision_function_.argmax(axis=1)

        # Calculate confusion matrix
        tn, fp, fn, tp = confusion_matrix(
```

```
            y, oob_predictions, labels=[0, 1]
        ).ravel()

        # Record model performance
        results.append({
            "ntree": ntree,
            "mtry": str(mtry),
            "OOB_accuracy": accuracy_score(y, oob_predictions),
            "e1_precision": precision_score(y, oob_predictions, pos_label=1),
            "e1_recall": recall_score(y, oob_predictions, pos_label=1),
            "e1_F1": f1_score(y, oob_predictions, pos_label=1),
            "non_e1_precision": precision_score(y, oob_predictions, pos_label=0),
            "non_e1_recall": recall_score(y, oob_predictions, pos_label=0),
            "non_e1_F1": f1_score(y, oob_predictions, pos_label=0),
            "TN": tn,
            "FP": fp,
            "FN": fn,
            "TP": tp
        })

        print("Completed:", ntree, "trees, mtry =", mtry)

# Convert results into a table
results_df = pd.DataFrame(results)

# Display results
print("\nSix-model OOB comparison:")
print(results_df.to_string(index=False, float_format="%.5f"))

# Save results for Section 6
results_df.to_csv(
    r"C:\Users\enyes\OneDrive\Documents\HW2_OOB_results.csv",
    index=False
)

print("\nResults saved to HW2_OOB_results.csv")
```

HW2_OOB_results.csv is available in addition to this document.

6.3 Three-Fold, Manual Cross Validations Script

```
# HW2: Manual Three-Fold Cross-Validation
# Random Forest classifier for e1 neuronal cluster
# Selected parameters: ntree=100, mtry="sqrt"

import pandas as pd
import numpy as np

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix,
```

```
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

from pathlib import Path

# ------------------------------------------------
# 1. Load the training database
# ------------------------------------------------

folder = Path(r"C:\Users\enyes\OneDrive\Documents")

training_db = pd.read_csv(folder / "training_db.csv")

X = training_db.drop(columns=["Label"])
y = training_db["Label"]

print("Training database loaded")
print("Features:", X.shape)
print("Labels:", y.shape)

# ------------------------------------------------
# 2. Manually construct three stratified folds
# ------------------------------------------------

rng = np.random.default_rng(42)

# Find the row positions for each class
class_0_indices = np.where(y.to_numpy() == 0)[0]
class_1_indices = np.where(y.to_numpy() == 1)[0]

# Shuffle observations within each class
rng.shuffle(class_0_indices)
rng.shuffle(class_1_indices)

# Divide each class into three approximately equal parts
class_0_folds = np.array_split(class_0_indices, 3)
class_1_folds = np.array_split(class_1_indices, 3)

# Combine class subsets to construct stratified folds
folds = []

for i in range(3):
    fold = np.concatenate([
        class_0_folds[i],
        class_1_folds[i]
    ])

    rng.shuffle(fold)
    folds.append(fold)

print("\nThree stratified folds created")

for i, fold in enumerate(folds):
    print(
        "Fold", i + 1,
        "samples:", len(fold),
        "class counts:",
```

```
        y.iloc[fold].value_counts().to_dict()
    )

# Verify that all observations appear exactly once
all_indices = np.concatenate(folds)

assert len(all_indices) == len(training_db)
assert len(np.unique(all_indices)) == len(training_db)

# ------------------------------------------------
# 3. Train and evaluate each fold manually
# ------------------------------------------------

fold_results = []

# Store held-out predictions in original row order
all_predictions = np.full(len(y), -1, dtype=int)

for i in range(3):

    print("\nTraining fold", i + 1)

    # Current fold is the evaluation set
    test_indices = folds[i]

    # Remaining two folds form the training set
    train_indices = np.concatenate([
        folds[j] for j in range(3) if j != i
    ])

    X_train = X.iloc[train_indices]
    y_train = y.iloc[train_indices]

    X_test = X.iloc[test_indices]
    y_test = y.iloc[test_indices]

    # Construct a fresh Random Forest
    rf = RandomForestClassifier(
        n_estimators=100,
        max_features="sqrt",
        bootstrap=True,
        oob_score=True,
        random_state=42,
        n_jobs=-1
    )

    # Train only on the two training folds
    rf.fit(X_train, y_train)

    # Predict the held-out fold
    predictions = rf.predict(X_test)

    # Store predictions for pooled evaluation
    all_predictions[test_indices] = predictions

    # Calculate confusion matrix
    tn, fp, fn, tp = confusion_matrix(
        y_test, predictions, labels=[0, 1]
    ).ravel()
```

```
    # Calculate performance metrics
    fold_results.append({
        "fold": i + 1,
        "train_samples": len(train_indices),
        "test_samples": len(test_indices),
        "accuracy": accuracy_score(y_test, predictions),
        "e1_precision": precision_score(
            y_test, predictions, pos_label=1, zero_division=0
        ),
        "e1_recall": recall_score(
            y_test, predictions, pos_label=1, zero_division=0
        ),
        "e1_F1": f1_score(
            y_test, predictions, pos_label=1, zero_division=0
        ),
        "non_e1_precision": precision_score(
            y_test, predictions, pos_label=0, zero_division=0
        ),
        "non_e1_recall": recall_score(
            y_test, predictions, pos_label=0, zero_division=0
        ),
        "non_e1_F1": f1_score(
            y_test, predictions, pos_label=0, zero_division=0
        ),
        "TN": tn,
        "FP": fp,
        "FN": fn,
        "TP": tp
    })

    print("Fold", i + 1, "completed")
    print("Accuracy:", round(
        accuracy_score(y_test, predictions), 5
    ))
    print("Confusion matrix:")
    print(confusion_matrix(
        y_test, predictions, labels=[0, 1]
    ))

# ------------------------------------------------
# 4. Summarize the three folds
# ------------------------------------------------

results_df = pd.DataFrame(fold_results)

metric_columns = [
    "accuracy",
    "e1_precision",
    "e1_recall",
    "e1_F1",
    "non_e1_precision",
    "non_e1_recall",
    "non_e1_F1"
]

summary_df = pd.DataFrame({
    "mean": results_df[metric_columns].mean(),
    "std": results_df[metric_columns].std()
})
```

```
print("\nThree-fold CV results:")
print(results_df.to_string(
    index=False,
    float_format="%.5f"
))

print("\nMean and standard deviation:")
print(summary_df.to_string(
    float_format="%.5f"
))

# -----------------------------------------------
# 5. Pooled held-out evaluation
# -----------------------------------------------

assert np.all(all_predictions != -1)

pooled_cm = confusion_matrix(
    y, all_predictions, labels=[0, 1]
)

print("\nPooled CV confusion matrix:")
print(pooled_cm)

print("\nPooled CV accuracy:",
      format(accuracy_score(y, all_predictions), ".5f"))

print("Pooled CV e1 F1:",
      format(f1_score(y, all_predictions, pos_label=1), ".5f"))

print("Pooled CV non-e1 F1:",
      format(f1_score(y, all_predictions, pos_label=0), ".5f"))

# -----------------------------------------------
# 6. Export results
# -----------------------------------------------

results_df.to_csv(
    folder / "HW2_CV_fold_results.csv",
    index=False
)

summary_df.to_csv(
    folder / "HW2_CV_summary.csv"
)

pooled_cm_df = pd.DataFrame(
    pooled_cm,
    index=["Actual non-e1", "Actual e1"],
    columns=["Predicted non-e1", "Predicted e1"]
)

pooled_cm_df.to_csv(
    folder / "HW2_CV_confusion_matrix.csv"
)

print("\nAll CV results saved successfully!")
```

7.1 Feature Ranking Script

```
# HW2 Section 7: Random Forest Feature Ranking
# Gini-based feature importance

import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path
from sklearn.ensemble import RandomForestClassifier

# 1. Load training data

folder = Path(r"C:\Users\enyes\OneDrive\Documents")

training_db = pd.read_csv(folder / "training_db.csv")

X = training_db.drop(columns=["Label"])
y = training_db["Label"]

print("Training data loaded:", X.shape)

# 2. Train the selected Random Forest

rf = RandomForestClassifier(
    n_estimators=100,
    max_features="sqrt",
    bootstrap=True,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)

print("Training selected Random Forest...")
rf.fit(X, y)
print("Training complete!")

# 3. Extract Gini-based feature importance

importance_df = pd.DataFrame({
    "Gene": X.columns,
    "Gini_Importance": rf.feature_importances_
})

# Sort genes from highest to lowest importance
importance_df = importance_df.sort_values(
    by="Gini_Importance",
    ascending=False
).reset_index(drop=True)

# Add rank numbers
importance_df["Rank"] = importance_df.index + 1

# 4. Display top ten features

top_10 = importance_df.head(10)
```

```
print("\nTop 10 ranked genes:")
print(top_10.to_string(
    index=False,
    float_format="%.5f"
))

# 5. Compare with published e1 markers

reference_genes = [
    "TESPA1",
    "LINC00507",
    "SLC17A7",
    "KCNIP1"
]

print("\nPublished e1 marker gene rankings:")

for gene in reference_genes:
    result = importance_df[
        importance_df["Gene"] == gene
    ]

    if not result.empty:
        print(result.to_string(index=False))
    else:
        print(gene, "not found in feature list")

# 6. Create top-ten feature importance graph

plt.figure(figsize=(10, 6))

plt.barh(
    top_10["Gene"][::-1],
    top_10["Gini_Importance"][::-1]
)

plt.xlabel("Gini Feature Importance")
plt.ylabel("Gene")
plt.title("Top 10 Random Forest Features: e1 Classification")

plt.tight_layout()

plt.savefig(
    folder / "HW2_RF_Top10_Features.png",
    dpi=300
)

plt.show()

# 7. Export complete ranking

importance_df.to_csv(
    folder / "HW2_RF_Feature_Ranking.csv",
    index=False
)

print("\nFeature ranking saved successfully!")
print("Feature importance chart saved successfully!")
```

7.2 Reduced Feature Script

```
# ============================================================
# CSC 695/895 HW2 - Section 7
# Reduced-Feature Random Forest Experiment
#
# Model: 100 trees, max_features="sqrt"
# Method: Manual stratified three-fold cross-validation
# Feature selection: Gini importance within each training fold
#
# ChatGPT-assisted Python implementation
# ============================================================

import pandas as pd
import numpy as np

from pathlib import Path
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

# ------------------------------------------------------------
# 1. Load training database
# ------------------------------------------------------------

folder = Path(r"C:\Users\enyes\OneDrive\Documents")

training_db = pd.read_csv(folder / "training_db.csv")

X = training_db.drop(columns=["Label"])
y = training_db["Label"]

print("Training database loaded!")
print("Features:", X.shape)
print("Labels:", y.shape)

assert X.shape == (869, 608)
assert X.isna().sum().sum() == 0

# ------------------------------------------------------------
# 2. Manually create three stratified folds
# ------------------------------------------------------------

rng = np.random.default_rng(42)

class_0 = np.where(y.to_numpy() == 0)[0]
class_1 = np.where(y.to_numpy() == 1)[0]

rng.shuffle(class_0)
rng.shuffle(class_1)

parts_0 = np.array_split(class_0, 3)
```

```
parts_1 = np.array_split(class_1, 3)

folds = []

for i in range(3):

    fold = np.concatenate([
        parts_0[i],
        parts_1[i]
    ])

    rng.shuffle(fold)
    folds.append(fold)

# Verify each observation appears in exactly one fold
all_indices = np.concatenate(folds)

assert len(all_indices) == len(y)
assert len(np.unique(all_indices)) == len(y)

print("\nThree stratified folds created successfully!")

for i, fold in enumerate(folds):
    print(
        "Fold", i + 1,
        "samples:", len(fold),
        "class counts:",
        y.iloc[fold].value_counts().to_dict()
    )

# ------------------------------------------------------------
# 3. Define feature-count experiments
# ------------------------------------------------------------

feature_counts = [4, 6, 10, 608]

results = []
selected_gene_records = []

# ------------------------------------------------------------
# 4. Run reduced-feature experiments
# ------------------------------------------------------------

for i in range(3):

    print("\n====================================")
    print("Evaluating fold", i + 1)
    print("====================================")

    test_idx = folds[i]

    train_idx = np.concatenate([
        folds[j] for j in range(3) if j != i
    ])

    X_train = X.iloc[train_idx]
    X_test = X.iloc[test_idx]

    y_train = y.iloc[train_idx]
    y_test = y.iloc[test_idx]
```

```
    # Rank genes using training observations ONLY
    ranking_rf = RandomForestClassifier(
        n_estimators=100,
        max_features="sqrt",
        bootstrap=True,
        random_state=42,
        n_jobs=-1
    )

    ranking_rf.fit(X_train, y_train)

    gene_ranking = pd.Series(
        ranking_rf.feature_importances_,
        index=X_train.columns
    ).sort_values(ascending=False)

    # Test four feature counts using the same fold
    for n_features in feature_counts:

        print(
            "\nFold", i + 1,
            "- Testing", n_features, "features"
        )

        if n_features == X.shape[1]:
            # Full-feature baseline: retain original column order
            selected_genes = X.columns.tolist()
        else:
            # Select top genes from THIS training fold
            selected_genes = (
                gene_ranking.head(n_features).index.tolist()
            )

        # Record which genes were selected
        if n_features != X.shape[1]:
            for rank, gene in enumerate(selected_genes, start=1):
                selected_gene_records.append({
                    "fold": i + 1,
                    "feature_count": n_features,
                    "rank": rank,
                    "gene": gene
                })

        # Train a fresh Random Forest
        model = RandomForestClassifier(
            n_estimators=100,
            max_features="sqrt",
            bootstrap=True,
            random_state=42,
            n_jobs=-1
        )

        model.fit(
            X_train[selected_genes],
            y_train
        )

        # Predict held-out observations
        predictions = model.predict(
```

```
            X_test[selected_genes]
        )

        # Calculate confusion matrix
        tn, fp, fn, tp = confusion_matrix(
            y_test,
            predictions,
            labels=[0, 1]
        ).ravel()

        # Record performance metrics
        results.append({
            "features": n_features,
            "fold": i + 1,
            "train_samples": len(train_idx),
            "test_samples": len(test_idx),
            "accuracy": accuracy_score(
                y_test, predictions
            ),
            "e1_precision": precision_score(
                y_test, predictions,
                pos_label=1, zero_division=0
            ),
            "e1_recall": recall_score(
                y_test, predictions,
                pos_label=1, zero_division=0
            ),
            "e1_F1": f1_score(
                y_test, predictions,
                pos_label=1, zero_division=0
            ),
            "non_e1_F1": f1_score(
                y_test, predictions,
                pos_label=0, zero_division=0
            ),
            "TN": tn,
            "FP": fp,
            "FN": fn,
            "TP": tp
        })

        print(
            "Accuracy:",
            format(accuracy_score(y_test, predictions), ".5f")
        )

        print(
            "e1 F1:",
            format(
                f1_score(y_test, predictions, pos_label=1),
                ".5f"
            )
        )

# ------------------------------------------------------------
# 5. Create fold-level results table
# ------------------------------------------------------------

results_df = pd.DataFrame(results)
```

```
results_df = results_df.sort_values(
    by=["features", "fold"]
)

print("\n====================================")
print("REDUCED-FEATURE FOLD RESULTS")
print("====================================")

print(
    results_df.to_string(
        index=False,
        float_format="%.5f"
    )
)

# ------------------------------------------------------------
# 6. Calculate mean and standard deviation
# ------------------------------------------------------------

metric_columns = [
    "accuracy",
    "e1_precision",
    "e1_recall",
    "e1_F1",
    "non_e1_F1"
]

summary = results_df.groupby("features")[
    metric_columns
].agg(["mean", "std"])

print("\n====================================")
print("REDUCED-FEATURE SUMMARY")
print("====================================")

print(
    summary.to_string(
        float_format="%.5f"
    )
)

# ------------------------------------------------------------
# 7. Create pooled confusion matrices
# ------------------------------------------------------------

pooled_results = []

for n_features in feature_counts:

    subset = results_df[
        results_df["features"] == n_features
    ]

    tn = int(subset["TN"].sum())
    fp = int(subset["FP"].sum())
    fn = int(subset["FN"].sum())
    tp = int(subset["TP"].sum())

    total = tn + fp + fn + tp
```

```
    pooled_results.append({
        "features": n_features,
        "TN": tn,
        "FP": fp,
        "FN": fn,
        "TP": tp,
        "total": total,
        "pooled_accuracy": (tp + tn) / total,
        "pooled_e1_F1": (
            2 * tp / (2 * tp + fp + fn)
        )
    })

pooled_df = pd.DataFrame(pooled_results)

print("\n====================================")
print("POOLED CONFUSION MATRIX RESULTS")
print("====================================")

print(
    pooled_df.to_string(
        index=False,
        float_format="%.5f"
    )
)

# ------------------------------------------------------------
# 8. Export experimental results
# ------------------------------------------------------------

results_df.to_csv(
    folder / "HW2_Reduced_Feature_Folds.csv",
    index=False
)

summary.to_csv(
    folder / "HW2_Reduced_Feature_Summary.csv"
)

pooled_df.to_csv(
    folder / "HW2_Reduced_Feature_Pooled.csv",
    index=False
)

selected_gene_df = pd.DataFrame(selected_gene_records)

selected_gene_df.to_csv(
    folder / "HW2_Reduced_Feature_Genes.csv",
    index=False
)

print("\nSection 7 experiment complete!")
print("Four result files saved successfully.")
print("Verification database remains untouched.")
```

7.3 Reduced Feature Comparison

```
Training database loaded!
Features: (869, 608)
Labels: (869,)

Three stratified folds created successfully!
Fold 1 samples: 291 class counts: {0: 191, 1: 100}
Fold 2 samples: 289 class counts: {0: 190, 1: 99}
Fold 3 samples: 289 class counts: {0: 190, 1: 99}

==================================
Evaluating fold 1
==================================

Fold 1 - Testing 4 features
Accuracy: 0.99656
e1 F1: 0.99502

Fold 1 - Testing 6 features
Accuracy: 0.98625
e1 F1: 0.98000

Fold 1 - Testing 10 features
Accuracy: 0.98625
e1 F1: 0.98020

Fold 1 - Testing 608 features
Accuracy: 0.99313
e1 F1: 0.99000

==================================
Evaluating fold 2
==================================

Fold 2 - Testing 4 features
Accuracy: 0.98270
e1 F1: 0.97487

Fold 2 - Testing 6 features
Accuracy: 0.99308
e1 F1: 0.98990

Fold 2 - Testing 10 features
Accuracy: 0.99308
e1 F1: 0.98990

Fold 2 - Testing 608 features
Accuracy: 0.99654
e1 F1: 0.99497

==================================
Evaluating fold 3
==================================

Fold 3 - Testing 4 features
Accuracy: 0.98616
e1 F1: 0.97980
```

```
Fold 3 - Testing 6 features
Accuracy: 0.99308
e1 F1: 0.98990

Fold 3 - Testing 10 features
Accuracy: 0.98962
e1 F1: 0.98492

Fold 3 - Testing 608 features
Accuracy: 0.99654
e1 F1: 0.99497

==================================
REDUCED-FEATURE FOLD RESULTS
==================================
 features  fold  train_samples  test_samples  accuracy  e1_precision  e1_recall   e1_F1
non_e1_F1  TN  FP  FN  TP
        4     1            578           291   0.99656       0.99010    1.00000 0.99502
0.99738 190   1   0 100
        4     2            580           289   0.98270       0.97000    0.97980 0.97487
0.98681 187   3   2  97
        4     3            580           289   0.98616       0.97980    0.97980 0.97980
0.98947 188   2   2  97
        6     1            578           291   0.98625       0.98000    0.98000 0.98000
0.98953 189   2   2  98
        6     2            580           289   0.99308       0.98990    0.98990 0.98990
0.99474 189   1   1  98
        6     3            580           289   0.99308       0.98990    0.98990 0.98990
0.99474 189   1   1  98
       10     1            578           291   0.98625       0.97059    0.99000 0.98020
0.98947 188   3   1  99
       10     2            580           289   0.99308       0.98990    0.98990 0.98990
0.99474 189   1   1  98
       10     3            580           289   0.98962       0.98000    0.98990 0.98492
0.99208 188   2   1  98
      608     1            578           291   0.99313       0.99000    0.99000 0.99000
0.99476 190   1   1  99
      608     2            580           289   0.99654       0.99000    1.00000 0.99497
0.99736 189   1   0  99
      608     3            580           289   0.99654       0.99000    1.00000 0.99497
0.99736 189   1   0  99

==================================
REDUCED-FEATURE SUMMARY
==================================
         accuracy         e1_precision         e1_recall           e1_F1         non_e1_F1
             mean     std         mean     std      mean     std    mean     std      mean
std
features
4         0.98847 0.00722      0.97997 0.01005   0.98653 0.01166 0.98323 0.01051   0.99122
0.00550
6         0.99080 0.00394      0.98660 0.00572   0.98660 0.00572 0.98660 0.00572   0.99300
0.00301
10        0.98965 0.00341      0.98016 0.00966   0.98993 0.00006 0.98501 0.00485   0.99210
0.00263
608       0.99540 0.00197      0.99000 0.00000   0.99667 0.00577 0.99332 0.00287   0.99650
0.00150

==================================
```

```
POOLED CONFUSION MATRIX RESULTS
===================================
 features  TN  FP  FN  TP  total  pooled_accuracy  pooled_e1_F1
        4 565   6   4 294    869          0.98849       0.98328
        6 567   4   4 294    869          0.99079       0.98658
       10 565   6   3 295    869          0.98964       0.98497
      608 568   3   1 297    869          0.99540       0.99331

Section 7 experiment complete!
Four result files saved successfully.
Verification database remains untouched.
```

8.1 Run Time Engine

```
# ============================================================
# CSC 695/895 HW2 - Section 8
# Six-Feature Random Forest Runtime Engine
#
# Model: 100 trees, max_features="sqrt"
# Training: 869 observations
# Verification: 2 previously withheld observations
#
# ChatGPT-assisted Python implementation
# ============================================================

import pandas as pd
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier

# ------------------------------------------------------------
# 1. Load training and verification databases
# ------------------------------------------------------------

folder = Path(r"C:\Users\enyes\OneDrive\Documents")

training_db = pd.read_csv(folder / "training_db.csv")
verification_db = pd.read_csv(folder / "verification_db.csv")

print("Training database:", training_db.shape)
print("Verification database:", verification_db.shape)

# ------------------------------------------------------------
# 2. Specify selected six genes
# ------------------------------------------------------------

selected_genes = [
    "SLC17A7",
    "KCNIP1",
    "TESPA1",
    "LINC00152",
    "SFTA1P",
```

```
    "LINC00507"
]

# Check that all selected genes exist in both databases
assert all(gene in training_db.columns for gene in selected_genes)
assert all(gene in verification_db.columns for gene in selected_genes)

# Separate features and ground-truth labels
X_train = training_db[selected_genes]
y_train = training_db["Label"]

X_verify = verification_db[selected_genes]
y_verify = verification_db["Label"]

print("\nSelected genes:", selected_genes)
print("Training features:", X_train.shape)
print("Verification features:", X_verify.shape)

# ------------------------------------------------------------
# 3. Train the final Random Forest classifier
# ------------------------------------------------------------

runtime_rf = RandomForestClassifier(
    n_estimators=100,
    max_features="sqrt",
    bootstrap=True,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)

print("\nTraining six-feature runtime classifier...")

runtime_rf.fit(X_train, y_train)

print("Runtime classifier training complete!")

# ------------------------------------------------------------
# 4. Predict the two verification observations
# ------------------------------------------------------------

predictions = runtime_rf.predict(X_verify)
probabilities = runtime_rf.predict_proba(X_verify)

# Locate the probability column for each class
class_0_index = list(runtime_rf.classes_).index(0)
class_1_index = list(runtime_rf.classes_).index(1)

# ------------------------------------------------------------
# 5. Compare predictions with ground truth
# ------------------------------------------------------------

results = pd.DataFrame({
    "Ground_Truth": y_verify.to_numpy(),
    "Predicted_Class": predictions,
    "P_non_e1": probabilities[:, class_0_index],
    "P_e1": probabilities[:, class_1_index],
    "Correct": predictions == y_verify.to_numpy()
})
```

```
print("\n====================================")
print("FINAL RUNTIME VERIFICATION RESULTS")
print("====================================")

print(results.to_string(
    index=False,
    float_format="%.5f"
))

# ------------------------------------------------------------
# 6. Export results
# ------------------------------------------------------------

results.to_csv(
    folder / "HW2_Runtime_Verification.csv",
    index=False
)

print("\nRuntime verification results saved!")
print("Section 8 experiment complete!")
```